## Extra Credit Problems

Here are a number of problems which can be worked for extra credit. Each of these problems, if done correctly, will add one point each to your total grade for the semester. In other words, if your grade for the semester is 88 and you correctly work three extra credit problems, you will have a final grade of 91. This can mean the difference between a B+ and an A-, but only if you are willing to do the work!

To present more of a challenge, these problems are *NOT* in your current textbook. Rather, they are from the texts that were used in previous years. In addition, some of them are not in *ANY* textbook, but rather are made up for you from scratch.

Your extra credit submissions are due in your repository at the beginning of class on the Wednesday of week 14, so you'll have an idea of how many, if any, you may want to do by that time.

The extra credit problems are:

## Problem 3.10, Rajlich page 46

Write a class that supports course grading; it contains an array of 30 students such that each student is identified by an integer and has a course grade. There is also a method that can change the student grade and a method that computes the grade point average for the entire class of 30 students.

```
public class Student {
   private int id;
   private double grade;

   public Student(int id) {
      this.id = id;
      this.grade = 0.0;
   }

   public int getId() { return id; }
```

```
    public double getGrade() { return grade; }
    public void setGrade(double grade) { this.grade = grade; }
}

public class CourseGrading {
    private Student[] students;

    public CourseGrading(int[] studentIds) {
    students = new Student[30];
    for (int i = 0; i < 30; i++) {
        students[i] = new Student(studentIds[i]);
    }
}



public boolean changeGrade(int studentId, double newGrade) {
    for (Student s : students) {
        if (s != null && s.getId() == studentId) {
            s.setGrade(newGrade);
            return true; // Return true if successful
        }
    }
    System.out.println("Student ID " + studentId + " not found.");
    return false; // Return false if not found
}

    public double computeGPA() {
        double sum = 0.0;
        for (Student s : students) {
            sum += s.getGrade();
        }
        return sum / students.length;
    }
}
```

## Problem 4.2, Rajlich page 63

Draw a class diagram of a library lending books using the following classes: Librarian, Lending Session, Overdue Fine, Book Inventory, Book, Library, Checkout System, and Library Card. Download and install a UML tool such as StarUML, ArgoUML, or Poseidon and use the tool to draw the class diagram.

Library

+ inventory: BookInventory

+ checkoutSystem: CheckoutSystem

+ addBook(b: Book)

+ registerCard(c: LibraryCard)

BookInventory

+ books: List<Book>

+ addBook(b: Book)

+ removeBook(isbn: String): Book

Book

- title: String

- author: String

- isbn: String

+ getTitle(): String

LibraryCard

- cardId: String

- owner: String

+ getCardId(): String

Librarian

- name: String

+ assistCheckout(card: LibraryCard, book: Book): LendingSession

CheckoutSystem

+ processCheckout(book: Book, card: LibraryCard): LendingSession

LendingSession

- checkoutDate: Date

- returnDate: Date

- book: Book

- card: LibraryCard

+ calculateFine(returnedDate: Date): OverdueFine

OverdueFine

- amount: double

+ payFine()

- Library 1 — 1 BookInventory
- Library 1 — 1 CheckoutSystem
- BookInventory 1 — * Book
- CheckoutSystem 1 — * LendingSession
- LendingSession 1 — 1 Book
- LendingSession 1 — 1 LibraryCard
- Librarian 1 — * CheckoutSystem (if multiple librarians)
- LendingSession 1 — 0..1 OverdueFine

## Problem 4.3, Rajlich page 63

Draw an activity diagram of pumping gas and paying by credit card at the pump. Include at least five activities, such as "Select fuel grade" and at least two decisions, such as "Get receipt?" Download and install a UML tool such as StarUML, ArgoUML, or Poseidon and use the tool to draw the diagram. If you did this for the previous problem, you may use the same tool.

Start

[Customer]

:Insert card;

:Enter PIN;

[Pump System]

:Validate card;

-->|invalid| :Reject card;

-->|valid| :Select fuel grade;

:Select fuel amount or fill;

:Lift nozzle;

:Pump fuel;

:Return nozzle;

:Print receipt?;

-->|yes| :Print receipt;

-->|no|

:Remove card;

Stop

## Problem 10.6, Schach page 166

For the following program design a minimal test suite:

```
            public static void main(String args[]) {
                int i,j;
                int k = 0;
                i = Integer.parseInt(args[0]);
                j = Integer.parseInt(args[1]);
                if( i > 30 ) {
                   if( i <= 60 && j <= 150 )
                      k = 1;
                   else if( i <= 90 && j <= 150 )
                      k = 2;
                   else
                      k = 3;
                   }
                if( i == j && i <= 30 )
                   k=4;
                System.out.println( k );
             }
```

```
assert(run(20, 10) == 0);
```

```
assert(run(20, 20) == 4);

assert(run(50, 100) == 1);

assert(run(70, 100) == 2);

assert(run(100, 100) == 3);

assert(run(50, 200) == 3);
```

## Problem 10.8, Schach page 167

In test driven development, a test is written and then the code to pass it. Given the following test, write a method that passes the test.

```
            public class TestGrade {
                Grade testGrade;
                public void testGetFinalGrade() {
                assert( testGrade.getFinalGrade(70) == "Pass" );
                assert( testGrade.getFinalGrade(69) == "Fail" );
             };

public class Grade {

      public String getFinalGrade(int score) {

        if (score >= 70) return "Pass";

        return "Fail";

      }
```

## Problem 10.8, Schach page 167

Expand the test in exercise 10.8 to include grades A to F using the following grade scale:

| Letter Grade | Minimum Percentage |
|---|---|
| A | 90 |
| B | 80 |
| C | 70 |
| D | 60 |
| F | Less than 60 |

```
public class Grade {

    public String getFinalGrade(int score) {

        if (score >= 90) return "A";

        else if (score >= 80) return "B";

        else if (score >= 70) return "C";

        else if (score >= 60) return "D";

        else return "F";

    }

}

@Test
```

```
public void testBoundaries() {

    assertEquals("A", grade.getFinalGrade(90));

    assertEquals("B", grade.getFinalGrade(80));

    assertEquals("C", grade.getFinalGrade(70));

    assertEquals("D", grade.getFinalGrade(60));

    assertEquals("F", grade.getFinalGrade(59));

    assertEquals("F", grade.getFinalGrade(0));

}
```

## Problem 11.2, Schach page 388

Instructor note: Although this problem is actually not from your text book for this semester, you should still be able to answer it, based on your knowledge of software engineering that you gained during the course.

Consider the following recipe for grilled pockwester.

Ingredients:

- 1 large onion
- 1 can of frozen orange juice
- Freshly squeezed juice of 1 lemon
- 1 cup bread crumbs
- Flour
- Milk
- 3 medium-sized shallots
- 2 medium-sized eggplants
- 1 fresh pockwester
- ½ cup Pouilly Fuissé
- 1 garlic

- Parmesan cheese
- 4 free-range eggs

The night before, take one lemon, squeeze it, strain the juice, and freeze it. Take one large onion and three shallots, dice them, and grill them in a skillet. when clouds of black smoke start to come off, add 2 cups of fresh orange juice. Stir vigorously. Slice the lemon into paper-thin slices and add to the mixture. In the meantime, coat the mushrooms in flow, dip them in milk, and then shake them in a paper bag with the bread crumbs. In a saucepan, heat ½ cup of Pouilly Fuissé. when it reaches 170°, add the sugar and continue to heat. When the sugar has caramelized, add the mushrooms. Blend the mixture for 10 minutes or until all the lumps have been removed. Add the eggs. Now take the pockwester, and kill it by sprinkling it with frobs. Skin the pockwester, break it into bite-sized chunks, and add it to the mixture. Bring to a boil and simmer, uncovered. The eggs previously should have been vigorously stirred with a wire whisk for 5 minutes. When the pockwester is soft to the touch, place it on a serving platter, sprinkle with Parmesan cheese, and broil for not more than 4 minutes.

Determine the ambiguities, omissions, and contradictions in the preceding specification. Don't forget to check the ingredients list for such things as well. You may either write your findings in a descriptive paragraph, or you may make a list. (For the record, a pockwester is an imaginary sort of fish and *frobs* is slang for generic hors d'oeuvres.)

## Ambiguities

- “fresh pockwester” — size, preparation state unclear
- “flour” vs “flow” (typo/undefined ingredient)
- “frob” is undefined quantity
- “add sugar” — sugar not in ingredient list
- “blend mixture” — what mixture exactly?
- “heat until 170°” — Celsius or Fahrenheit?
- “until soft to touch” — no objective measure
- “grill until black smoke” — unsafe / undefined endpoint

## Omissions

- No cooking times for most steps
- No temperature units
- No serving size
- No order clarity between simultaneous steps
- No quantity for garlic, Parmesan, flour, milk

## Contradictions

- Ingredients list includes mushrooms, but recipe uses “pockwester” as fish and later treats like meat
- Frozen lemon juice then fresh lemon slices (inconsistent prep state)
- “Blend for 10 minutes” after frying/boiling sequence unclear
- Eggs added twice (mixed earlier and added later)

### Flowchart Problem [worth 2 points]

Download and install the Raptor Flowchart Runner. Experiment with it until you can work it, at least in a rudimentary fashion. Then, implement the following program using the flowchart runner. When it works, take a screen shot of your flowchart and embed the graphic in a document for submission. Also submit the resulting code run from the tool. Here is the program (should look familiar from CMSI 185 or CMSI 186):

```
            function fizzbuzzdisplay() {
                for( var j = 1; j < 11; j++ ) {
                   for( var i = j; i < 101; (i += 10) ) {
                      if( !(i % 3) ) {
                         output += "FIZZ\n";
                         if( !(i % 5) ) {
                            output += "BUZZ\n";
                         }
                      } else if( !(i % 5) ) {
                         output += "BUZZ\n";
                      } else {
                         output += i;
                      }
                   }
                }
                alert( output );
             }
```

## Software Configuration Management Plan [worth 2 points]

Write a Configuration Management Plan for your project. Use the information found on the course docs page for the Software Configuration Management Plan. Perform an Internet search to find a CMMI example of the document, or the version which is made available by the SEI, and use it as a model. Make the document as complete as possible. Cite your source if you use something from the Internet as a model.

## Software Test Plan or Test Procedure

Write a Test Plan or Test Procedure for your project. Use the information found on the course docs page for the Software Configuration Management Plan. Perform an Internet search to find a CMMI example of the document, or the version which is made available by the SEI, and use it as a model. Make the document as complete as possible. Cite your source if you use something from the Internet as a model.po9